

passed to the `len()` function. By “ordered,” we mean that there is a first value, second value, and so on in the sequence. For example, the following two list values aren’t considered equal because their values are ordered differently:

---

```
>>> [1, 2, 3] == [3, 2, 1]
False
```

---

A *mapping* is an object of any container data type that uses keys instead of an index. A mapping can be ordered or unordered. Dictionaries in Python 3.4 and earlier are unordered because there is no first or last key-value pair in a dictionary:

---

```
>>> spam = {'a': 1, 'b': 2, 'c': 3, 'd': 4} # This is run from CPython 3.5.
>>> list(spam.keys())
['a', 'c', 'd', 'b']
>>> spam['e'] = 5
>>> list(spam.keys())
['e', 'a', 'c', 'd', 'b']
```

---

You have no guarantee of getting items in a consistent order from dictionaries in early versions of Python. As a result of dictionaries’ unordered nature, two dictionary literals written with different orders for their key-value pairs are still considered equal:

---

```
>>> {'a': 1, 'b': 2, 'c': 3} == {'c': 3, 'a': 1, 'b': 2}
True
```

---

But starting in CPython 3.6, dictionaries do retain the insertion order of their key-value pairs:

---

```
>>> spam = {'a': 1, 'b': 2, 'c': 3, 'd': 4} # This is run from CPython 3.6.
>>> list(spam)
['a', 'b', 'c', 'd']
>>> spam['e'] = 5
>>> list(spam)
['a', 'b', 'c', 'd', 'e']
```

---

This is a feature in the CPython 3.6 interpreter but not in other interpreters for Python 3.6. All Python 3.7 interpreters support ordered